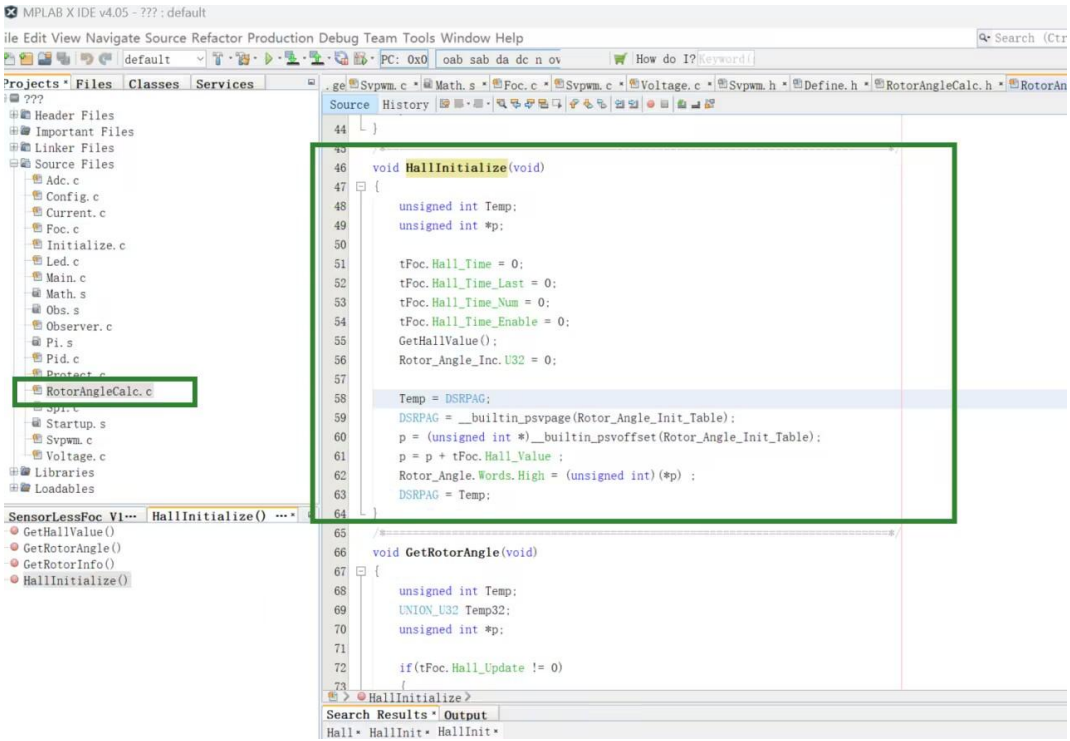
45-SVPWM 16

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。




```
/*==========================================================================*/
static void HallInitialize(void)
{
    unsigned int Temp;
    unsigned int *p;

    tBldc.Hall_Time = 0;
    tBldc.Hall_Time_Last = 0;
    tBldc.Hall_Time_Num = 0;
    tBldc.Hall_Time_Enable = 0;
    GetHallValue();
    Rotor_Angle_Inc.U32 = 0;

    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(Rotor_Angle_Init_Table);
    p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Init_Table);
    p = p + tBldc.Hall_Value ;
    Rotor_Angle.Words.High = (unsigned int)(*p) ;
    DSRPAG = Temp;
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

下面我们来看上节课内容对应的代码实现哈，大家看下这段代码啊，这是初始化的时候，读取霍尔组合值，并且确定初始转子位置角对应的代码，当然，里面呢，包含一些使用变量的初始化。

```
static void HallInitialize(void)
{
    unsigned int Temp;
    unsigned int *p;

    tBldc.Hall_Time = 0;
    tBldc.Hall_Time_Last = 0;
    tBldc.Hall_Time_Num = 0;
    tBldc.Hall_Time_Enable = 0;
    GetHallValue();
    Rotor_Angle_Inc.U32 = 0;

    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(Rotor_Angle_Init_Table);
    p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Init_Table);
    p = p + tBldc.Hall_Value ;
    Rotor_Angle.Words.High = (unsigned int)(*p) ;
    DSRPAG = Temp;
}
```

这几个变量呢，都是我们后面计算角度增量的时候用到的，这里呢，先初始化为 0。

```
static void HallInitialize(void)
{
    unsigned int Temp;
    unsigned int *p;

    tBldc.Hall_Time = 0;
    tBldc.Hall_Time_Last = 0;
    tBldc.Hall_Time_Num = 0;
    tBldc.Hall_Time_Enable = 0;
    GetHallValue();
    Rotor_Angle_Inc.U32 = 0;

    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(Rotor_Angle_Init_Table);
    p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Init_Table);
    p = p + tBldc.Hall_Value ;
    Rotor_Angle.Words.High = (unsigned int)(*p) ;
    DSRPAG = Temp;
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这个呢，表示转子位置角的增量，我们后面会有相应的公式来计算它。

```
static void HallInitialize(void)
{
    unsigned int Temp;
    unsigned int *p;

    tBldc.Hall_Time = 0;
    tBldc.Hall_Time_Last = 0;
    tBldc.Hall_Time_Num = 0;
    tBldc.Hall_Time_Enable = 0;
    GetHallValue();
    Rotor_Angle_Inc.U32 = 0;

    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(Rotor_Angle_Init_Table);
    p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Init_Table);
    p = p + tBldc.Hall_Value ;
    Rotor_Angle.Words.High = (unsigned int)(*p) ;
    DSRPAG = Temp;
}
```

接着呢，这个函数就是用来读取霍尔组合值。

```
static void GetHallValue(void)
{
    unsigned int i;
    UNION_WORD Temp;

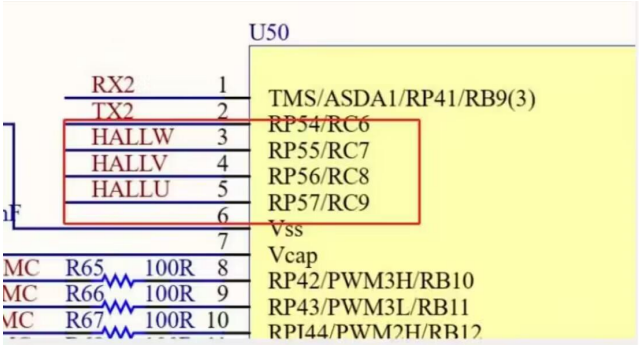
    do
    {
        i = 0;
        do
        {
            Temp.Word = 0;
            Temp.Bits.B2 = PORTCbits.RC9;
            Temp.Bits.B1 = PORTCbits.RC8;
            Temp.Bits.B0 = PORTCbits.RC7;
            tBldc.Hall_Value_Buf[i++] = Temp.Word;
            DELAY1US();
        }while(i < 3);
    }while((tBldc.Hall_Value_Buf[0] != Temp.Word) && (tBldc.Hall_Value_Buf[1] != Temp.Word));

    tBldc.Hall_Value = Temp.Word;
    if (tBldc.Hall_Value != tBldc.Hall_Value_Old)
    {
        tBldc.Hall_Value_Old = tBldc.Hall_Value;

        if ((tBldc.Hall_Value > 0) && (tBldc.Hall_Value < 7))
        {
            tBldc.Hall_Update = 1;
        }
        else
        {
            Error_Flag.Bits.Hall = 1;
        }
    }
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

大家来看下这段代码，霍尔信号有 3 路，我们要读取组合值的话，就要读 3 个 IO 的电平状态。



根据原理图呢，我们是知道跟霍尔信号相连的 IO 分别是 RC7　RC8　RC9。

```
/*=============================================================================*/
static void GetHallValue(void)
{
    unsigned int i;
    UNION_WORD Temp;

    do
    {
        i = 0;
        do
        {
            Temp.Word = 0;
            Temp.Bits.B2 = PORTCbits.RC9;
            Temp.Bits.B1 = PORTCbits.RC8;
            Temp.Bits.B0 = PORTCbits.RC7;
            tBldc.Hall_Value_Buf[i++] = Temp.Word;
            DELAY1US();
        }while(i < 3);
    }while((tBldc.Hall_Value_Buf[0] != Temp.Word) && (tBldc.Hall_Value_Buf[1] != Temp.Word));
```

所以，这里我们采取位读取的方式，获取 IO 端口的值。temp 变量的话，是用下面这个联合体类型定义的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
typedef union
{
    unsigned int Word;
    struct
    {
        unsigned B0:1;
        unsigned B1:1;
        unsigned B2:1;
        unsigned B3:1;
        unsigned B4:1;
        unsigned B5:1;
        unsigned B6:1;
        unsigned B7:1;
        unsigned B8:1;
        unsigned B9:1;
        unsigned B10:1;
        unsigned B11:1;
        unsigned B12:1;
        unsigned B13:1;
        unsigned B14:1;
        unsigned B15:1;
    }Bits;
}UNION_WORD;
```

这里读取呢，仅仅利用了 16 位中其中的 3 个位。

```
/*=====================================================================*/
static void GetHallValue(void)
{
    unsigned int i;
    UNION_WORD Temp;

    do
    {
        i = 0;
        do
        {
            Temp.Word = 0;
            Temp.Bits.B2 = PORTCbits.RC9;
            Temp.Bits.B1 = PORTCbits.RC8;
            Temp.Bits.B0 = PORTCbits.RC7;
            tBldc.Hall_Value_Buf[i++] = Temp.Word;
            DELAY1US();
        }while(i < 3);
    }while((tBldc.Hall_Value_Buf[0] != Temp.Word) && (tBldc.Hall_Value_Buf[1] != Temp.Word));
```

所以，红框中的代码，就是分别把 RC9　RC8　RC7 三个 IO 端口的电平状态存到 Word 中的 B2　B1　B0 这三个位中。

```
/*=====================================================================*/
static void GetHallValue(void)
{
    unsigned int i;
    UNION_WORD Temp;

    do
    {
        i = 0;
        do
        {
            Temp.Word = 0;
            Temp.Bits.B2 = PORTCbits.RC9;
            Temp.Bits.B1 = PORTCbits.RC8;
            Temp.Bits.B0 = PORTCbits.RC7;
            tBldc.Hall_Value_Buf[i++] = Temp.Word;
            DELAY1US();
        }while(i < 3);
    }while((tBldc.Hall_Value_Buf[0] != Temp.Word) && (tBldc.Hall_Value_Buf[1] != Temp.Word));
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

接着呢，是把读到的值，存到一个临时的 BUF 中。目的呢，就是要检测霍尔信号是否有干扰，或者说，要对霍尔信号进行滤波。这个霍尔滤波是怎么做的呢？

```
unsigned int i;
UNION_WORD Temp;

do
{
    i = 0;
    do
    {
        Temp.Word = 0;
        Temp.Bits.B2 = PORTCbits.RC9;
        Temp.Bits.B1 = PORTCbits.RC8;
        Temp.Bits.B0 = PORTCbits.RC7;
        tBldc.Hall_Value_Buf[i++] = Temp.Word;
        DELAY1US();
    }while(i < 3);
}while((tBldc.Hall_Value_Buf[0] != Temp.Word) && (tBldc.Hall_Value_Buf[1] != Temp.Word));
```

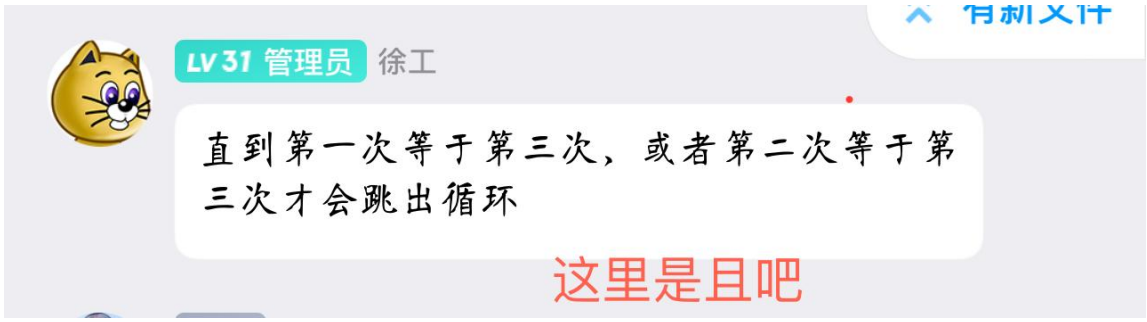
这里呢，是隔 1us，读取一次 IO 口状态，存到 BUF 数组中，连续间隔 1us 读 3 次。

```
unsigned int i;
UNION_WORD Temp;

do
{
    i = 0;
    do
    {
        Temp.Word = 0;
        Temp.Bits.B2 = PORTCbits.RC9;
        Temp.Bits.B1 = PORTCbits.RC8;
        Temp.Bits.B0 = PORTCbits.RC7;
        tBldc.Hall_Value_Buf[i++] = Temp.Word;
        DELAY1US();
    }while(i < 3);
}while((tBldc.Hall_Value_Buf[0] != Temp.Word) && (tBldc.Hall_Value_Buf[1] != Temp.Word));
```

接着呢，判断是否有异常值出现。这里采用的是第一次读取的值跟第三次读取值且第二次读取的值跟第三次读取值是否不相等判断。如果说确实有　第一次不等于第三次，第二次不等于第三次，就会在这个循环中一直判断。直到第一次等于第三次，或者第二次等于第三次才会跳出循环。也就是说，满足其中两次采样值相等，程序流就会继续往下执行，实现一个软件滤波。因为霍尔信号比较重要，受到干扰的话，霍尔值可能会出现问题。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



问：

答：跳出是或，不满足循环条件是或。

问：万一第三次霍尔变化了呢？

答：变化就继续循环采样，直到不变了。

问：为什么是 1us 时间呢？

答：这是经验值。

问：加个霍尔最大更新时间判断？

答：可以的，正常是没有问题的。

出问题的话，转子位置角就会计算错误，严重时电机电流过大甚至会失控。所以，必须确保读到的值是准确的。

```
do
{
    i = 0;
    do
    {
        Temp.Word = 0;
        Temp.Bits.B2 = PORTCbits.RC9;
        Temp.Bits.B1 = PORTCbits.RC8;
        Temp.Bits.B0 = PORTCbits.RC7;
        tBldc.Hall_Value_Buf[i++] = Temp.Word;
        DELAY1US();
    }while(i < 3);
}while((tBldc.Hall_Value_Buf[0] != Temp.Word) && (tBldc.Hall_Value_Buf[1] != Temp.Word));
```

所以，这段代码的作用，就是读取霍尔组合值，并进行了软件滤波处理。确保后面用于计算转子角度的值是可信的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
            Temp.Bits.B0 = PORTCbits.RC7;
            tBldc.Hall_Value_Buf[i++] = Temp.Word;
            DELAY1US();
        }while(i < 3);
    }while((tBldc.Hall_Value_Buf[0] != Temp.Word) && (tBldc.Hall_Value_Buf[1] != Temp.Word));

    tBldc.Hall_Value = Temp.Word;
    if (tBldc.Hall_Value != tBldc.Hall_Value_Old)
    {
        tBldc.Hall_Value_Old = tBldc.Hall_Value;

        if ((tBldc.Hall_Value > 0) && (tBldc.Hall_Value < 7))
        {
            tBldc.Hall_Update = 1;
        }
        else
        {
            Error_Flag.Bits.Hall = 1;
        }
    }
}
```

接着，只要跳出了循环，就说明前面的读取确保了至少两次读取结果是相同的。此时呢，把最后读取的值赋值给 Hall_Value 变量，作为当前霍尔组合值。下面呢，还有一些处理确保霍尔值在正常值范围内。

```
    tBldc.Hall_Value = Temp.Word;
    if (tBldc.Hall_Value != tBldc.Hall_Value_Old)
    {
        tBldc.Hall_Value_Old = tBldc.Hall_Value;

        if ((tBldc.Hall_Value > 0) && (tBldc.Hall_Value < 7))
        {
            tBldc.Hall_Update = 1;
        }
        else
        {
            Error_Flag.Bits.Hall = 1;
        }
    }
}
```

然后，还要判断当前的值跟之前的值是否不一样，不一样的话，才会更新霍尔值。如果说，霍尔值长时间跟 Old 值一样的话，可能霍尔信号异常，此时也需要报故障停机。

```
    tBldc.Hall_Value = Temp.Word;
    if (tBldc.Hall_Value != tBldc.Hall_Value_Old)
    {
        tBldc.Hall_Value_Old = tBldc.Hall_Value;

        if ((tBldc.Hall_Value > 0) && (tBldc.Hall_Value < 7))
        {
            tBldc.Hall_Update = 1;
        }
        else
        {
            Error_Flag.Bits.Hall = 1;
        }
    }
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

并且呢，虽然霍尔值更新了，还要确保值在 5 4 6 2 3 1 这 6 个值中，这是 120° 电角度安装的霍尔是这样。

```
tBldc.Hall_Value = Temp.Word;
if (tBldc.Hall_Value != tBldc.Hall_Value_Old)
{
    tBldc.Hall_Value_Old = tBldc.Hall_Value;

    if ((tBldc.Hall_Value > 0) && (tBldc.Hall_Value < 7))
    {
        tBldc.Hall_Update = 1;
    }
    else
    {
        Error_Flag.Bits.Hall = 1;
    }
}
```

确保霍尔值在正常值范围内，就说明这个霍尔值是有效值。否则的话，读到其它的值，也说明霍尔信号异常，异常的话，也可以报错。

```
static void GetHallValue(void)
{
    unsigned int i;
    UNION_WORD Temp;

    do
    {
        i = 0;
        do
        {
            Temp.Word = 0;
            Temp.Bits.B2 = PORTCbits.RC9;
            Temp.Bits.B1 = PORTCbits.RC8;
            Temp.Bits.B0 = PORTCbits.RC7;
            tBldc.Hall_Value_Buf[i++] = Temp.Word;
            DELAY10US();
        }while(i < 3);
    }while((tBldc.Hall_Value_Buf[0] != Temp.Word) && (tBldc.Hall_Value_Buf[1] != Temp.Word));

    tBldc.Hall_Value = Temp.Word;
    if (tBldc.Hall_Value != tBldc.Hall_Value_Old)
    {
        tBldc.Hall_Value_Old = tBldc.Hall_Value;

        if ((tBldc.Hall_Value > 0) && (tBldc.Hall_Value < 7))
        {
            tBldc.Hall_Update = 1;
        }
        else
        {
            Error_Flag.Bits.Hall = 1;
        }
    }
}
```

这就是这个读取霍尔值的函数，那么，读完之后呢，要根据读取的值来确定角度初始值。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
static void HallInitialize(void)
{
    unsigned int Temp;
    unsigned int *p;

    tBldc.Hall_Time = 0;
    tBldc.Hall_Time_Last = 0;
    tBldc.Hall_Time_Num = 0;
    tBldc.Hall_Time_Enable = 0;
    GetHallValue();
    Rotor_Angle_Inc.U32 = 0;

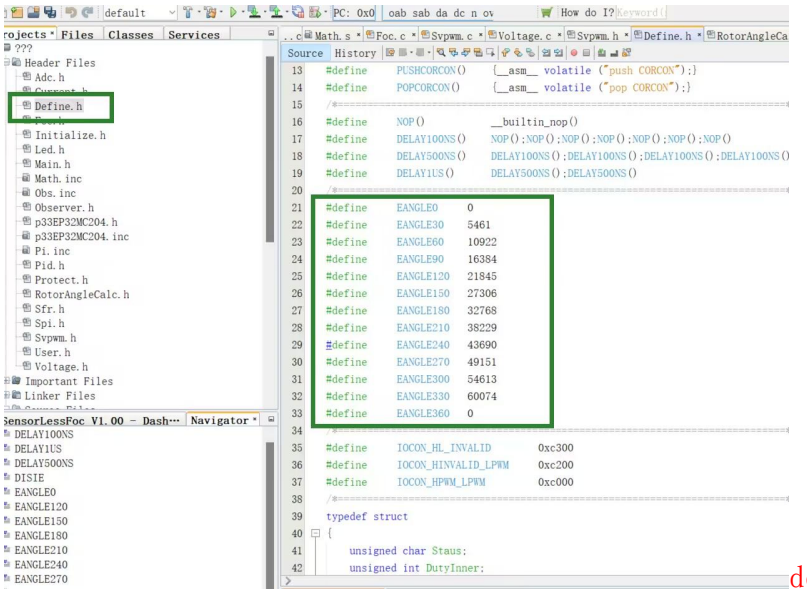
    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(Rotor_Angle_Init_Table);
    p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Init_Table);
    p = p + tBldc.Hall_Value ;
    Rotor_Angle.Words.High = (unsigned int)(*p) ;
    DSRPAG = Temp;
}
```

这里呢，把初始角度做了一个表。我们根据读到的霍尔值来查询。

```
const unsigned int  Rotor_Angle_Table_CCW[7] = {0, EANGLE150, EANGLE30, EANGLE90, EANGLE270, EANGLE210, EANGLE330};
const unsigned int  Rotor_Angle_Table_CW[7] = {0, EANGLE210, EANGLE90, EANGLE150, EANGLE330, EANGLE270, EANGLE30};
const unsigned int  Rotor_Angle_Init_Table[7] = {0, EANGLE180, EANGLE60, EANGLE120, EANGLE300, EANGLE240, EANGLE0};
```

这是根据我们之前讲的定义的。我们获取初始角度呢，就是根据霍尔值来查表。所以，查表之后，初始时的转子位置角就获取到了。关于这个霍尔的初始角度，大家有问题么？

问：表格里面的值没有看明白？



答：define.h 中有定义。

问：就是说正反转都用这个初始表？

答：是的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================================*/
static void HallInitialize(void)
{
    unsigned int Temp;
    unsigned int *p;

    tBldc.Hall_Time = 0;
    tBldc.Hall_Time_Last = 0;
    tBldc.Hall_Time_Num = 0;
    tBldc.Hall_Time_Enable = 0;
    GetHallValue();
    Rotor_Angle_Inc.U32 = 0;

    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(Rotor_Angle_Init_Table);
    p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Init_Table);
    p = p + tBldc.Hall_Value ;
    Rotor_Angle.Words.High = (unsigned int)(*p) ;
    DSRPAG = Temp;
}
```

上面初始化设置了一些初值，并根据初始时刻的霍尔值确定了初始的转子位置角，然后，我们需要考虑在转动过程中的角度求解。

```
/*==========================================================================*/
static void GetRotorInfo(void)
{
    GetHallValue();
    GetRotorAngle();
}
/*====================Copyright belongs to ZhangFeiDianZi====================*/
```

大家看，这里写了这样一个函数，就是获取转子位置信息。这个函数内部，调用了读取霍尔值的函数，也调用了获取转子角度的函数。

```
/*==========================================================================*/
static void GetRotorAngle(void)
{
    unsigned int Temp;
    UNION_U32 Temp32;
    unsigned int *p;

    if(tBldc.Hall_Update != 0)
    {
        tBldc.Hall_Update = 0;
        tBldc.Hall_Update_Count = 0;

        if (tBldc.Direction == 0x01)
        {
            Temp = DSRPAG;
            DSRPAG = __builtin_psvpage(Rotor_Angle_Table_CCW);
            p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Table_CCW);
            p = p + tBldc.Hall_Value;
            Rotor_Angle.Words.High = (unsigned int)(*p) ;
            DSRPAG = Temp;
        }
        else
        {
```

我们重点呢，是要来分析这个函数。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

首先呢，我们是需要根据每次霍尔值变化之后，确定出变化时的转子位置角。

（单片机口）　　逆时针　顺时针

HALLA	HALLB	HALLC	组合值	通电相序	转子位置	转子位置
1	0	1	2	b正 a负	30°	90°
1	0	0	3	c正 a负	90°	150°
1	1	0	1	c正 b负	150°	210°
0	1	0	5	a正 b负	210°	270°
0	1	1	4	a正 c负	270°	330°
0	0	1	6	b正 c负	330°	30°

也就是说，需要先把这 6 个角度确定出来，然后呢，再计算 60 度区间内的角度增量。

```
static void GetRotorAngle(void)
{
    unsigned int Temp;
    UNION_U32 Temp32;
    unsigned int *p;

    if(tBldc.Hall_Update != 0)
    {
        tBldc.Hall_Update = 0;
        tBldc.Hall_Update_Count = 0;

        if (tBldc.Direction == 0x01)
        {
            Temp = DSRPAG;
            DSRPAG = __builtin_psvpage(Rotor_Angle_Table_CCW);
            p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Table_CCW);
            p = p + tBldc.Hall_Value;
            Rotor_Angle.Words.High = (unsigned int)(*p) ;
            DSRPAG = Temp;
        }
        else
```

大家应该还记得，我们只有读到正确的霍尔值，这个 tBldc.Hall_Update 才会变为 1 变为 1 之后，也就说明霍尔值有更新了。有更新的话，我们就要根据更新的霍尔值来确定对应的角度。

```
if(tBldc.Hall_Update != 0)
{
    tBldc.Hall_Update = 0;
    tBldc.Hall_Update_Count = 0;

    if (tBldc.Direction == 0x01)
    {
        Temp = DSRPAG;
        DSRPAG = __builtin_psvpage(Rotor_Angle_Table_CCW);
        p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Table_CCW);
        p = p + tBldc.Hall_Value;
        Rotor_Angle.Words.High = (unsigned int)(*p) ;
        DSRPAG = Temp;
    }
    else
    {
        Temp = DSRPAG;
        DSRPAG = __builtin_psvpage(Rotor_Angle_Table_CW);
        p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Table_CW);
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这里清零的话，是为下一个霍尔更新做准备的。

```
if(tBldc.Hall_Update != 0)
{
    tBldc.Hall_Update = 0;
    tBldc.Hall_Update_Count = 0;

    if (tBldc.Direction == 0x01)
    {
        Temp = DSRPAG;
        DSRPAG = __builtin_psvpage(Rotor_Angle_Table_CCW);
        p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Table_CCW);
        p = p + tBldc.Hall_Value;
        Rotor_Angle.Words.High = (unsigned int)(*p) ;
        DSRPAG = Temp;
    }
    else
    {
```

问：这里查表代码，有啥优点？为什么不直接用数组加下标？

答：物理访问时间不一样，因为我们现在看到的 C 语言，看不出执行时间。如果改成汇编你就会知道时间不一样了。

问：这样更快一点？

答：是的，就像访问 RAM 一样快，可以关注手册中的单片机 PSV 功能。

然后，这个值的话，就是我们讲的，很长时间霍尔值都没有正常更新，设置了一个计数变量，计数变量到对应的计数值之后，还没有更新，就会给一个故障提示。比如说，堵转了，或者霍尔信号异常了，根据故障提示，就可以做停机处理。

```
if(tBldc.Hall_Update != 0)
{
    tBldc.Hall_Update = 0;
    tBldc.Hall_Update_Count = 0;

    if (tBldc.Direction == 0x01)
    {
        Temp = DSRPAG;
        DSRPAG = __builtin_psvpage(Rotor_Angle_Table_CCW);
        p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Table_CCW);
        p = p + tBldc.Hall_Value;
        Rotor_Angle.Words.High = (unsigned int)(*p) ;
        DSRPAG = Temp;
    }
    else
    {
        Temp = DSRPAG;
        DSRPAG = __builtin_psvpage(Rotor_Angle_Table_CW);
        p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Table_CW);
        p = p + tBldc.Hall_Value ;
        Rotor_Angle.Words.High = (unsigned int)(*p) ;
        DSRPAG = Temp;
    }
```

接着，这里就是分转动方向，来获取与霍尔值对应的转子角度，其实，也就是根据不同的方向查不同的霍尔值表。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
if(tBldc.Hall_Update != 0)
{
    tBldc.Hall_Update = 0;
    tBldc.Hall_Update_Count = 0;

    if (tBldc.Direction == 0x01)
    {
        Temp = DSRPAG;
        DSRPAG = __builtin_psvpage(Rotor_Angle_Table_CCW);
        p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Table_CCW);
        p = p + tBldc.Hall_Value;
        Rotor_Angle.Words.High = (unsigned int)(*p) ;
        DSRPAG = Temp;
    }
    else
    {
        Temp = DSRPAG;
        DSRPAG = __builtin_psvpage(Rotor_Angle_Table_CW);
        p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Table_CW);
        p = p + tBldc.Hall_Value ;
        Rotor_Angle.Words.High = (unsigned int)(*p) ;
        DSRPAG = Temp;
    }
```

也就是说，这个角度变量，就是更新之后的角度了。不像是初始那样，有 30° 误差的，这个角度是相对准确的。

```
if(tBldc.Hall_Update != 0)
{
    tBldc.Hall_Update = 0;
    tBldc.Hall_Update_Count = 0;

    if (tBldc.Direction == 0x01)
    {
        Temp = DSRPAG;
        DSRPAG = __builtin_psvpage(Rotor_Angle_Table_CCW);
        p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Table_CCW);
        p = p + tBldc.Hall_Value;
        Rotor_Angle.Words.High = (unsigned int)(*p) ;
        DSRPAG = Temp;
    }
    else
    {
        Temp = DSRPAG;
        DSRPAG = __builtin_psvpage(Rotor_Angle_Table_CW);
        p = (unsigned int *)__builtin_psvoffset(Rotor_Angle_Table_CW);
        p = p + tBldc.Hall_Value ;
        Rotor_Angle.Words.High = (unsigned int)(*p) ;
        DSRPAG = Temp;
    }
    Rotor_Angle.Words.Low = 0;
```

那么，这里把沿变化的时候，对应的角度获取到了，下面呢，就要获取 60° 的时间，根据时间来求解角度增量。求解到角度增量之后呢，就可以求得 60° 区间内的角度了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    }
    Rotor_Angle.Words.Low = 0;
    if(IFS0bits.IC1IF)
    {
        Temp = IC2BUF;
        Temp = IC3BUF;
        IFS0bits.IC1IF = 0;
        IFS0bits.IC2IF = 0;
        IFS2bits.IC3IF = 0;
        tBldc.Hall_Time = IC1BUF;
    }
    else if(IFS0bits.IC2IF)
    {
        Temp = IC1BUF;
        Temp = IC3BUF;
        IFS0bits.IC1IF = 0;
        IFS0bits.IC2IF = 0;
        IFS2bits.IC3IF = 0;
        tBldc.Hall_Time = IC2BUF;
    }
    else if(IFS2bits.IC3IF)
    {
        Temp = IC1BUF;
        Temp = IC2BUF;
        IFS0bits.IC1IF = 0;
        IFS0bits.IC2IF = 0;
        IFS2bits.IC3IF = 0;
        tBldc.Hall_Time = IC3BUF;
    }
```

大家看，这里的三个判断，就是判断是哪个沿发生了变化。因为是三路霍尔，三路霍尔不会同时有沿变化，每隔 60°，只会有其中一个沿变化。所以呢，这里就是检测一下是哪个沿发生变化，目的呢，就是要获取到此时的捕捉定时器的计数值。不同的沿，对应的捕捉定时器存储的值在不同的寄存器中。所以呢，需要判断出来，是哪个沿变化了，才能准确读到对应的计数值。

```
if(IFS0bits.IC1IF)
{
    Temp = IC2BUF;
    Temp = IC3BUF;
    IFS0bits.IC1IF = 0;
    IFS0bits.IC2IF = 0;
    IFS2bits.IC3IF = 0;
    tBldc.Hall_Time = IC1BUF;
}
```

大家看，假设是 IC1 发生了一个捕捉事件，此时呢，我们就要读取 IC1BUF 得到捕捉定时器的值。同时呢，也顺便对其它的捕捉模块进行清零处理。

```
else if(IFS0bits.IC2IF)
{
    Temp = IC1BUF;
    Temp = IC3BUF;
    IFS0bits.IC1IF = 0;
    IFS0bits.IC2IF = 0;
    IFS2bits.IC3IF = 0;
    tBldc.Hall_Time = IC2BUF;
}
else if(IFS2bits.IC3IF)
{
    Temp = IC1BUF;
    Temp = IC2BUF;
    IFS0bits.IC1IF = 0;
    IFS0bits.IC2IF = 0;
    IFS2bits.IC3IF = 0;
    tBldc.Hall_Time = IC3BUF;
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

IC2 跟 IC3 发生捕捉事件的时候，也是一样的处理方式。那么，这样的话，就把发生捕捉事件时的定时器的计数值拿到了。

大家看，这个 if 时 IC1 发生捕捉事件。我们读 IC3BUF IC2BUF 存放到 temp 中，是想对这两个 BUF 清 0. 清 0 的目的呢，是为了后面它们发生捕捉事件时，我们去读对应的值。这里的这个 temp 就是一个临时量，只是一个读操作，结果不会使用；这个 BUF 你可以看成是 FIFO。它是移动的。读一次，数据就移动一次。当里面只有一个数据的时候，读一次，里面的数据就没了。这个 BUF 我们不能写，只读，读清 0。

好，我们继续来看啊

```
else if(IFS0bits.IC2IF)
{
    Temp = IC1BUF;
    Temp = IC3BUF;
    IFS0bits.IC1IF = 0;
    IFS0bits.IC2IF = 0;
    IFS2bits.IC3IF = 0;
    tBldc.Hall_Time = IC2BUF;
}
else if(IFS2bits.IC3IF)
{
    Temp = IC1BUF;
    Temp = IC2BUF;
    IFS0bits.IC1IF = 0;
    IFS0bits.IC2IF = 0;
    IFS2bits.IC3IF = 0;
    tBldc.Hall_Time = IC3BUF;
}
if(tBldc.Hall_Time >= tBldc.Hall_Time_Last)
{
    Temp = tBldc.Hall_Time - tBldc.Hall_Time_Last;
}
else
{
    Temp = 65536 - tBldc.Hall_Time_Last + tBldc.Hall_Time;
}
tBldc.Hall_Time_Last = tBldc.Hall_Time;
```

大家这个时候来看下这段代码，也就是我们之前讲过的，求解 60° 的时间处理的思路当这个沿读到的时间大于上一个沿的时间的时候，60° 的时间直接就是当前的时间减去之前的时间。当现在读到的时间小于之前的时间，就要特殊处理一下。

```
}
if(tBldc.Hall_Time >= tBldc.Hall_Time_Last)
{
    Temp = tBldc.Hall_Time - tBldc.Hall_Time_Last;
}
else
{
    Temp = 65536 - tBldc.Hall_Time_Last + tBldc.Hall_Time;
}
tBldc.Hall_Time_Last = tBldc.Hall_Time;
```

然后呢，肯定要把当前的时间更新到之前的变量，为下一个沿更新做准备。那么这样的话，这个 Temp 变量中，就相当于存储了 60° 的计数差值。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
if (Temp != 0)
{
    tBldc.Hall_Time_Buf[tBldc.Hall_Time_Num] = Temp;

    if (tBldc.Hall_Time_Num < 6)
    {
        tBldc.Hall_Time_Num ++;
    }

    if(tBldc.Hall_Time_Num == 6)
    {
        tBldc.Hall_Time_Num = 0;
        tBldc.Hall_Time_Enable = 1;
    }

    if (tBldc.Hall_Time_Enable != 0)
    {
        Temp32.U32 = (unsigned long int)(tBldc.Hall_Time_Buf[0]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[1]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[2]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[3]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[4]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[5]);
        tBldc.Hall_Time = __builtin_divud(Temp32.U32,6);
    }
```

大家来看，Temp 是 60° 的计数差值。如果正常发生捕捉事件后，这个变量肯定就不是 0 了，这样的话，就会进入这个 if 分支。

```
if (Temp != 0)
{
    tBldc.Hall_Time_Buf[tBldc.Hall_Time_Num] = Temp;

    if (tBldc.Hall_Time_Num < 6)
    {
        tBldc.Hall_Time_Num ++;
    }

    if(tBldc.Hall_Time_Num == 6)
    {
        tBldc.Hall_Time_Num = 0;
        tBldc.Hall_Time_Enable = 1;
    }

    if (tBldc.Hall_Time_Enable != 0)
    {
        Temp32.U32 = (unsigned long int)(tBldc.Hall_Time_Buf[0]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[1]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[2]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[3]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[4]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[5]);
        tBldc.Hall_Time = __builtin_divud(Temp32.U32,6);
    }
```

那么，这大段代码是在干嘛呢？实际上，这里是对 60° 的计数差值做一个平均滤波。这样的话，这个差值会相对平滑些。后续计算出来的角度呢，也会相对平滑些 。处理好滤波之后呢，就可以计算角度增量了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    }

    if (tBldc.Hall_Time_Enable != 0)
    {
        Temp32.U32 = (unsigned long int)(tBldc.Hall_Time_Buf[0]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[1]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[2]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[3]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[4]);
        Temp32.U32 += (unsigned long int)(tBldc.Hall_Time_Buf[5]);
        tBldc.Hall_Time = __builtin_divud(Temp32.U32,6);
    }
    else
    {
        if (tBldc.Hall_Time_Num == 1)
            tBldc.Hall_Time = 27397;//50RPM
        else
            tBldc.Hall_Time = Temp;
    }

    if (tBldc.Hall_Time <= 342) //4000RPM
        tBldc.Hall_Time = 342;
    Rotor_Angle_Inc.Words.High = __builtin_divmodud(ANG_INC_COEF,tBldc.Hall_Time,&Rotor_Angle_Inc.Words.Low);
  }
}
```

这个 else 分支，处理的就是滤波值还没有更新的时候。没有更新之前呢，都是使用直接计算的差值。并且，还没有 2 个霍尔沿的时候呢，60 度的差值，我们按照最小的转速对应的值。那么，这个 50RPM 对应的计数差值怎么计算呢？其实是有对应的公式的

$$RPM = \frac{60\ 000\ 000us}{Hall_Time*4.27us*6*P}$$

捕捉模块的计数时钟，我们之前讲过是 Fp/256 对吧？ P 是极对数。这样，大家可以计算一下，RPM = 50 的时候，计数差值是多少啊？这里算完就是 23419，当已经有了两个沿更新后，就使用 Temp 就行。

```
    }
    else
    {
        if (tBldc.Hall_Time_Num == 1)
            tBldc.Hall_Time = 23419;//50RPM
        else
            tBldc.Hall_Time = Temp;
    }

    if (tBldc.Hall_Time <= 585) //2000RPM
        tBldc.Hall_Time = 585;
    Rotor_Angle_Inc.Words.High = __builtin_divmodud(ANG_INC_COEF,tBldc.Hall_Time,&Rotor_Angle_Inc.Words.Low);
  }
}
```

然后，因为我们电机额定转速是 2000，所以，最小的计数差值我们也可以限制一下。这样的话，60° 的计数差值，就处理好了。下面呢，就可以根据公式，把角度增量给计算出来。

问：这个 4.27 咋算的？

答： 定时器输入频率 60MHz/256，16.67ns x 256 = 4.27us。

```
        tBldc.Hall_Time = __builtin_divud(Temp32.U32,6);
    }
    else
    {
        if (tBldc.Hall_Time_Num == 1)
            tBldc.Hall_Time = 23419;//50RPM
        else
            tBldc.Hall_Time = Temp;
    }

    if (tBldc.Hall_Time <= 585) //2000RPM
        tBldc.Hall_Time = 585;
    Rotor_Angle_Inc.Words.High = __builtin_divmodud(ANG_INC_COEF,tBldc.Hall_Time,&Rotor_Angle_Inc.Words.Low);
  }
}
else
{
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

也就是说，这句话就是计算角度增量。

$$\Delta\theta r = \frac{\Delta t}{T} * 60度 = \frac{50}{\frac{256}{60} * temp} * \frac{60}{360} * 65536 = \frac{50 * \frac{60}{256} * \frac{60}{360} * 65536}{temp}$$

计算的公式是这个

$$\Delta\theta r = \frac{\Delta t}{T} * 60度 = \frac{50}{\frac{256}{60} * temp} * \frac{60}{360} * 65536 = \frac{50 * \frac{60}{256} * \frac{60}{360} * 65536}{temp} = \boxed{\frac{128000}{temp}}$$

```
/*=============================================================================*/
#define     ANG_INC_COEF    128000  //ANG_INC_COEF = 60 / 360 * 65536 * 50us * 60 /256
/*=============================================================================*/
```

所以，这里用宏定义把这个 128000　用一个符号代替了。

```
        }

        if (tBldc.Hall_Time <= 585) //2000RPM
            tBldc.Hall_Time = 585;
        Rotor_Angle_Inc.Words.High = __builtin_divmodud(ANG_INC_COEF, tBldc.Hall_Time, &Rotor_Angle_Inc.Words.Low);
    }
}
else
```

这里呢，计算用到了一个内建函数的除法，这个除法呢，还把除法的余数给收集起来了。

__builtin_divmodud

说明：

发出 16 位架构的本机无符号除法支持。值得注意的是，如果商无法装入 16 位结果中，结果（包括余数）将是非预期的。这种形式的内建函数会同时捕捉商和余数。

原型：

```
unsigned int __builtin_divmodud(
unsigned long dividend, unsigned int divisor,
unsigned int *remainder);
```

参数：

dividend 被除数
divisor 除数
remainder 指向余数的指针

返回值：

商和余数。

汇编操作符 / 机器指令：

divmodud

错误消息：

无

这是这个内建函数的原型。这个余数呢，我们放到转子角增量的低 16 位，参与 32 位的角度累加。这样的话，角度精度稍微高一些。这就回答了为什么角度增量要定义成 32 位的变量。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
        else
        {
            if (tBldc.Hall_Time_Num == 1)
                tBldc.Hall_Time = 23419;//50RPM
            else
                tBldc.Hall_Time = Temp;
        }

        if (tBldc.Hall_Time <= 585) //2000RPM
            tBldc.Hall_Time = 585;
        Rotor_Angle_Inc.Words.High = __builtin_divmodud(ANG_INC_COEF, tBldc.Hall_Time, &Rotor_Angle_Inc.Words.Low);
    }
}
else
{
    if (tBldc.Hall_Update_Count < 65535)                    //65535 x 50us = 3.276750s
```

所以，到目前为止，这个角度增量就求解出来了。

```
            Rotor_Angle_Inc.Words.High = __builtin_divmodud(ANG_INC_COEF, tBldc.Hall_Time, &Rotor_Angle_Inc.
        }
    }
    else
    {
        if (tBldc.Hall_Update_Count < 65535)                    //65535 x 50us = 3.276750s
        {
            tBldc.Hall_Update_Count ++;

            if (tBldc.Direction == 0x01)
                Rotor_Angle.U32 += Rotor_Angle_Inc.U32;
            else
                Rotor_Angle.U32 -= Rotor_Angle_Inc.U32;
        }
        else
        {
            if (Motor_Flag.Bits.Status)
                Error_Flag.Bits.Hall = 1;
            else
                tBldc.Hall_Update_Count = 0;
        }
    }
    tBldc.Eangle_Rotor = Rotor_Angle.Words.High;
}
```

然后呢，这个 else 就是没有霍尔沿更新的时候，我们要利用求解出来的角度增量，来计算 60° 区间内的转子位置角。

```
else
{
    if (tBldc.Hall_Update_Count < 65535)                    //65535 x 50us = 3.276750s
    {
        tBldc.Hall_Update_Count ++;

        if (tBldc.Direction == 0x01)
            Rotor_Angle.U32 += Rotor_Angle_Inc.U32;
        else
            Rotor_Angle.U32 -= Rotor_Angle_Inc.U32;
    }
    else
    {
        if (Motor_Flag.Bits.Status)
            Error_Flag.Bits.Hall = 1;
        else
            tBldc.Hall_Update_Count = 0;
    }
}
tBldc.Eangle_Rotor = Rotor_Angle.Words.High;
```

大家看，这里对应这个计数变量，这个计数变量呢，就是霍尔值没有更新的时候会递增计数。正常情况下，肯定不会计数到最大值。一旦有霍尔沿变化，他就会被清 0。如果长时间不更新霍尔沿的话，就说明霍尔异常，或者堵转。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：刚开始启动时　没有角度增量怎么办？
答：按最低速度计算。

```
else
{
    if (tBldc.Hall_Update_Count < 65535)                    //65535 x 50us = 3.276750s
    {
        tBldc.Hall_Update_Count ++;

        if (tBldc.Direction == 0x01)
            Rotor_Angle.U32 += Rotor_Angle_Inc.U32;
        else
            Rotor_Angle.U32 -= Rotor_Angle_Inc.U32;
    }
    else
    {
        if (Motor_Flag.Bits.Status)
            Error_Flag.Bits.Hall = 1;
        else
            tBldc.Hall_Update_Count = 0;
    }
}
```

此时呢，就会进入到这个 else 分支，提示霍尔故障。如果正常的话，就会在 if 分支内。

```
}
else
{
    if (tBldc.Hall_Update_Count < 65535)                    //65535 x 50us = 3.276750s
    {
        tBldc.Hall_Update_Count ++;

        if (tBldc.Direction == 0x01)
            Rotor_Angle.U32 += Rotor_Angle_Inc.U32;
        else
            Rotor_Angle.U32 -= Rotor_Angle_Inc.U32;
    }
    else
    {
        if (Motor_Flag.Bits.Status)
            Error_Flag.Bits.Hall = 1;
        else
            tBldc.Hall_Update_Count = 0;
    }
}
tBldc.Eangle_Rotor = Rotor_Angle.Words.High;
}
```

在这个分支内的话，就会根据旋转方向，计算转子位置角。这里呢，就是 32 位累加了，高 16 位是角度增量的整数部分，低 16 位是角度增量的小数部分。

```
}
else
{
    if (tBldc.Hall_Update_Count < 65535)                    //65535 x 50us = 3.276750s
    {
        tBldc.Hall_Update_Count ++;

        if (tBldc.Direction == 0x01)
            Rotor_Angle.U32 += Rotor_Angle_Inc.U32;
        else
            Rotor_Angle.U32 -= Rotor_Angle_Inc.U32;
    }
    else
    {
        if (Motor_Flag.Bits.Status)
            Error_Flag.Bits.Hall = 1;
        else
            tBldc.Hall_Update_Count = 0;
    }
}
tBldc.Eangle_Rotor = Rotor_Angle.Words.High;
}
```

然后，最终转子位置角呢，我们还是取(0.16)格式的，所以取 32 位的高 16 位。这样的话，就把 360° 范围内的转子位置角给求解出来了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================================
static void GetRotorInfo(void)
{
    GetHallValue();
    GetRotorAngle();
}
/*====================Copyright belongs to ZhangFeiDianZi=====================
```

这个函数呢，就是求取转子位置角对应的函数，后面会调用它。
今天内容就讲到这里。大家早点休息，谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。